

🏰 ДОМЕН 10: ТЕХНИЧЕСКОЕ ЛИДЕРСТВО — ПОЛНАЯ ДЕТАЛИЗАЦИЯ (ОТ НУЛЯ ДО ЭКСПЕРТА)

СТРУКТУРА КАЖДОЙ ТЕМЫ

Для каждой темы я даю:

- **Что нужно знать** — теория для понимания
 - **Вопросы на собеседовании** (Junior → Senior → Staff)
 - **Практика** — конкретные задания для закрепления
-

РАЗДЕЛ 10.1: КОММУНИКАЦИЯ И ДОКУМЕНТАЦИЯ (JUNIOR → MIDDLE)

10.1.1. Техническая документация (Junior Level)

№ 10.1.1.1 — README

- **Что нужно знать:** структура (описание проекта, установка, запуск, API, лицензия)
- **Вопросы на собеседовании:** "Как написать хороший README?"
- **Практика:** Написать README для своего pet-проекта

№ 10.1.1.2 — Комментарии в коде

- **Что нужно знать:** когда писать, а когда не надо (чистый код сам себя документирует)
- **Вопросы на собеседовании:** "Когда нужно писать комментарии?"
- **Практика:** Провести code review своего кода и добавить комментарии там, где логика неочевидна

№ 10.1.1.3 — Документирование API

- **Что нужно знать:** OpenAPI/Swagger, JSDoc, Doxygen
- **Вопросы на собеседовании:** "Как документировать API?"
- **Практика:** Добавить Swagger в [ASP.NET](#) Core проект

№ 10.1.1.4 — ADR (Architecture Decision Record)

- **Что нужно знать:** структура (Context, Decision, Consequences)
- **Вопросы на собеседовании:** "Что такое ADR и зачем он нужен?"
- **Практика:** Написать ADR для выбора БД в проекте

№ 10.1.1.5 — Wiki

- **Что нужно знать:** ведение внутренней документации (Confluence, Notion, GitHub Wiki)
 - **Вопросы на собеседовании:** "Как организовать документацию в команде?"
 - **Практика:** Создать Wiki для своего проекта с разделами: архитектура, инструкции, FAQ
-

10.1.2. Коммуникация (Middle Level)

№ 10.1.2.1 — Как задавать вопросы

- **Что нужно знать:** синдром "Googleable" (не задавать вопросы, ответ на которые можно найти за 5 минут)
- **Вопросы на собеседовании:** "Как правильно задавать вопросы?"
- **Практика:** Написать вопрос для своего тимлида с контекстом и своими попытками решения

№ 10.1.2.2 — Активное слушание

- **Что нужно знать:** перефразирование, уточнение, эмпатия
- **Вопросы на собеседовании:** "Что такое активное слушание?"
- **Практика:** На созвоне с коллегой перефразировать его мысль, чтобы убедиться, что понял правильно

№ 10.1.2.3 — Презентации

- **Что нужно знать:** как доносить технические решения до нетехнической аудитории (продакты, менеджеры)
- **Вопросы на собеседовании:** "Как объяснить техническое решение продакту?"
- **Практика:** Подготовить презентацию на 5 минут о своем проекте для продакта

№ 10.1.2.4 — Письменная коммуникация

- **Что нужно знать:** email, Slack/Telegram, формат сообщений
- **Вопросы на собеседовании:** "Как написать сообщение в Slack, чтобы на него ответили?"

- **Практика:** Написать сообщение коллеге с четким вопросом и контекстом

№ 10.1.2.5 — Аргументация

- **Что нужно знать:** как отстаивать свою точку зрения на технических обсуждениях
 - **Вопросы на собеседовании:** "Как убедить команду использовать ваш подход?"
 - **Практика:** Подготовить аргументацию для внедрения нового инструмента (например, перехода на Go)
-

10.1.3. Практические задания (Junior → Middle Level)

№ 10.1.3.1

- **Задание:** Написать README для своего проекта с установкой, запуском, примером API
- **Что проверяет:** Знание документации

№ 10.1.3.2

- **Задание:** Написать ADR для выбора NoSQL vs SQL в своем проекте
- **Что проверяет:** Знание архитектурных решений

№ 10.1.3.3

- **Задание:** Подготовить презентацию на 5 минут о своем проекте для нетехнической аудитории
 - **Что проверяет:** Навыки коммуникации
-

РАЗДЕЛ 10.2: CODE REVIEW И КАЧЕСТВО КОДА (JUNIOR → SENIOR)

10.2.1. Code Review (Junior → Middle Level)

№ 10.2.1.1 — Что проверять

- **Что нужно знать:** архитектура, читаемость, производительность, безопасность, тесты
- **Вопросы на собеседовании:** "Что вы проверяете на code review?"
- **Практика:** Провести code review пул-реквеста коллеги

№ 10.2.1.2 — Модель SBI

- **Что нужно знать:** Situation-Behavior-Impact для фидбека
- **Вопросы на собеседовании:** "Как давать фидбек, чтобы не обидеть?"
- **Практика:** Написать feedback коллеге по модели SBI

№ 10.2.1.3 — Конструктивная критика

- **Что нужно знать:** как критиковать код, а не человека
- **Вопросы на собеседовании:** "Как критиковать код, чтобы не обидеть?"
- **Практика:** Написать комментарий к коду: что плохо, почему и как улучшить

№ 10.2.1.4 — Прием критики

- **Что нужно знать:** как реагировать на замечания, не принимая их на свой счет
- **Вопросы на собеседовании:** "Как реагировать на критику в code review?"
- **Практика:** Ответить на критику своего кода: поблагодарить, согласиться или аргументировать

№ 10.2.1.5 — Соглашения

- **Что нужно знать:** единый стиль кода (editorconfig, clang-format, ESLint)
- **Вопросы на собеседовании:** "Как поддерживать единый стиль кода в команде?"
- **Практика:** Настроить clang-format для C++ проекта и добавить в CI

№ 10.2.1.6 — Time-to-merge

- **Что нужно знать:** код не должен висеть в ревью больше дня
- **Вопросы на собеседовании:** "Как ускорить процесс code review?"
- **Практика:** Написать правило в команде: ревью должно быть в течение 24 часов

10.2.2. Качество кода (Middle → Senior Level)

№ 10.2.2.1 — Технический долг

- **Что нужно знать:** как управлять, как приоритезировать
- **Вопросы на собеседовании:** "Что такое технический долг?"
- **Практика:** Создать бэклог технического долга для своего проекта

№ 10.2.2.2 — Рефакторинг

- **Что нужно знать:** признаки (запахи) кода, когда делать
- **Вопросы на собеседовании:** "Когда нужно делать рефакторинг?"
- **Практика:** Выполнить рефакторинг одного из модулей: избавиться от дублирования, улучшить читаемость

№ 10.2.2.3 — Тестирование

- **Что нужно знать:** unit, integration, e2e, покрытие кода
- **Вопросы на собеседовании:** "Какие виды тестов вы используете?"
- **Практика:** Добавить unit-тесты для модуля с покрытием > 80%

№ 10.2.2.4 — Статический анализ

- **Что нужно знать:** линтеры (SonarQube, clang-tidy, ESLint)
- **Вопросы на собеседовании:** "Как автоматизировать проверку качества кода?"
- **Практика:** Настроить SonarQube в CI и исправить все критичные проблемы

№ 10.2.2.5 — Автоматизация

- **Что нужно знать:** pre-commit хуки, пайплайны для проверок
- **Вопросы на собеседовании:** "Как автоматизировать проверки перед коммитом?"
- **Практика:** Настроить pre-commit хук для проверки форматирования

10.2.3. Практические задания (Junior → Senior Level)

№ 10.2.3.1

- **Задание:** Провести code review пул-реквеста коллеги (можно на GitHub) и оставить 10 комментариев
- **Что проверяет:** Знание code review

№ 10.2.3.2

- **Задание:** Создать бэклог технического долга для своего проекта
- **Что проверяет:** Управление техническим долгом

№ 10.2.3.3

- **Задание:** Настроить линтер для C++ проекта (clang-tidy) и исправить все warnings
- **Что проверяет:** Знание статического анализа

РАЗДЕЛ 10.3: ПРОЦЕССЫ РАЗРАБОТКИ (MIDDLE → SENIOR)

10.3.1. Agile и методологии (Middle Level)

№ 10.3.1.1 — Scrum

- **Что нужно знать:** роли (Product Owner, Scrum Master, Team), ритуалы (планирование, ежедневки, обзор, ретроспектива)
- **Вопросы на собеседовании:** "Что такое Scrum?"
- **Практика:** Провести планирование спринта для своей команды

№ 10.3.1.2 — Kanban

- **Что нужно знать:** доски, колонки, WIP-лимиты (ограничение на работу в процессе)
- **Вопросы на собеседовании:** "В чем разница между Scrum и Kanban?"
- **Практика:** Настроить Kanban-доску в Trello/Jira для своего проекта

№ 10.3.1.3 — Оценка задач

- **Что нужно знать:** Story Points, Planning Poker, T-shirt sizing
- **Вопросы на собеседовании:** "Как оценивать задачи?"
- **Практика:** Оценить задачи на следующий спринт с командой

№ 10.3.1.4 — Спринты

- **Что нужно знать:** итерации длиной 1-4 недели, цели спринта
- **Вопросы на собеседовании:** "Что такое спринт?"
- **Практика:** Сформулировать цель для следующего спринта

№ 10.3.1.5 — Ретроспектива

- **Что нужно знать:** анализ того, что пошло хорошо, что плохо, что улучшить

- **Вопросы на собеседовании:** "Как проводить ретроспективу?"
- **Практика:** Провести ретроспективу команды и записать 3 пункта для улучшения

№ 10.3.1.6 — Definition of Done

- **Что нужно знать:** критерии готовности задачи (код, тесты, документация, ревью)
 - **Вопросы на собеседовании:** "Что такое Definition of Done?"
 - **Практика:** Согласовать Definition of Done для своей команды
-

10.3.2. Управление проектами (Senior Level)

№ 10.3.2.1 — Дорожная карта (Roadmap)

- **Что нужно знать:** планирование на 3-6 месяцев, приоритеты
- **Вопросы на собеседовании:** "Как составить roadmap?"
- **Практика:** Составить roadmap для своего проекта на 3 месяца

№ 10.3.2.2 — Декомпозиция

- **Что нужно знать:** разбиение большой задачи на подзадачи
- **Вопросы на собеседовании:** "Как разбить эпик на задачи?"
- **Практика:** Разбить эпик "Внедрение аутентификации" на конкретные задачи

№ 10.3.2.3 — Приоритизация

- **Что нужно знать:** MoSCoW (Must-have, Should-have, Could-have, Won't-have)
- **Вопросы на собеседовании:** "Как приоритезировать задачи?"
- **Практика:** Приоритезировать бэклог задач по MoSCoW

№ 10.3.2.4 — Риски

- **Что нужно знать:** идентификация, оценка, mitigation
- **Вопросы на собеседовании:** "Как управлять рисками?"
- **Практика:** Провести анализ рисков для нового проекта

№ 10.3.2.5 — Стейкхолдеры

- **Что нужно знать:** кто они, как с ними работать (продакты, бизнес, клиенты)
- **Вопросы на собеседовании:** "Как работать с продактом?"
- **Практика:** Провести встречу с продактом для уточнения требований

№ 10.3.2.6 — Отчеты

- **Что нужно знать:** статус проектов, метрики, диаграммы Ганта
- **Вопросы на собеседовании:** "Как отчитываться перед руководством?"
- **Практика:** Подготовить отчет о статусе проекта для руководства

10.3.3. Практические задания (Middle → Senior Level)

№ 10.3.3.1

- **Задание:** Составить roadmap для своего проекта на 3 месяца
- **Что проверяет:** Планирование

№ 10.3.3.2

- **Задание:** Разбить эпик на задачи и оценить их
- **Что проверяет:** Декомпозиция

№ 10.3.3.3

- **Задание:** Провести ретроспективу команды (хотя бы с собой) и записать 3 улучшения
- **Что проверяет:** Процессы

РАЗДЕЛ 10.4: НАЙМ И РАЗВИТИЕ (SENIOR → STAFF)

10.4.1. Найм (Senior Level)

№ 10.4.1.1 — Проведение собеседований

- **Что нужно знать:** System Design интервью vs Coding интервью
- **Вопросы на собеседовании:** "Как проводить техническое собеседование?"
- **Практика:** Составить план собеседования для Senior C++ разработчика

№ 10.4.1.2 — Тестовые задания

- **Что нужно знать:** как составлять, что проверять (алгоритмы, архитектура, качество кода)
- **Вопросы на собеседовании:** "Как составить тестовое задание?"
- **Практика:** Написать тестовое задание для Middle разработчика

№ 10.4.1.3 — Оценка кандидата

- **Что нужно знать:** hard skills, soft skills, культурный фит
- **Вопросы на собеседовании:** "Как оценить кандидата?"
- **Практика:** Составить чек-лист для оценки кандидата

№ 10.4.1.4 — Обратная связь

- **Что нужно знать:** как давать кандидату фидбек после собеседования
- **Вопросы на собеседовании:** "Как дать обратную связь кандидату?"
- **Практика:** Написать feedback для кандидата после технического интервью

№ 10.4.1.5 — Джоб-оффер

- **Что нужно знать:** как аргументировать предложение
- **Вопросы на собеседовании:** "Как подготовить оффер для кандидата?"
- **Практика:** Подготовить оффер-пакет для кандидата

10.4.2. Развитие команды (Staff Level)

№ 10.4.2.1 — Карьерные лестницы

- **Что нужно знать:** Junior → Middle → Senior → Lead → Staff
- **Вопросы на собеседовании:** "Как выстроить карьерный путь для разработчика?"
- **Практика:** Нарисовать карьерный путь для C++ разработчика в вашей компании

№ 10.4.2.2 — Индивидуальный план развития (IDP)

- **Что нужно знать:** как составлять, как отслеживать прогресс
- **Вопросы на собеседовании:** "Как составить IDP для подчиненного?"
- **Практика:** Составить IDP для джуниора в своей команде

№ 10.4.2.3 — Менторство

- **Что нужно знать:** как помогать расти младшим разработчикам
- **Вопросы на собеседовании:** "Как быть хорошим ментором?"
- **Практика:** Провести 1-on-1 с джуниором и обсудить его план развития

№ 10.4.2.4 — Обучение

- **Что нужно знать:** курсы, книги, конференции, внутренние митапы
- **Вопросы на собеседовании:** "Как организовать обучение в команде?"
- **Практика:** Организовать технический митап в команде

№ 10.4.2.5 — Knowledge Sharing

- **Что нужно знать:** как делиться знаниями в команде
- **Вопросы на собеседовании:** "Как организовать обмен знаниями?"
- **Практика:** Завести канал в Slack для технических статей

№ 10.4.2.6 — Передача знаний

- **Что нужно знать:** документация, онбординг, shadowing
- **Вопросы на собеседовании:** "Как передать знания новому сотруднику?"
- **Практика:** Составить онбординг-план для новичка

10.4.3. Практические задания (Senior → Staff Level)

№ 10.4.3.1

- **Задание:** Составить план собеседования для Senior C++ разработчика
- **Что проверяет:** Знание найма

№ 10.4.3.2

- **Задание:** Составить IDP для разработчика (реального или вымышленного)
- **Что проверяет:** Развитие команды

№ 10.4.3.3

- **Задание:** Составить онбординг-план для нового разработчика (первая неделя/месяц)
 - **Что проверяет:** Передача знаний
-

РАЗДЕЛ 10.5: СТРАТЕГИЯ И АРХИТЕКТУРА (STAFF LEVEL)

10.5.1. Архитектурное мышление (Staff Level)

№ 10.5.1.1 — NFR (Non-functional requirements)

- **Что нужно знать:** SLA, SLO, SLI (доступность, производительность, безопасность)
- **Вопросы на собеседовании:** "Что такое SLO?"
- **Практика:** Написать SLO для своего сервиса (доступность 99.95%)

№ 10.5.1.2 — Управление рисками

- **Что нужно знать:** идентификация, оценка, mitigation
- **Вопросы на собеседовании:** "Как управлять рисками в проекте?"
- **Практика:** Провести анализ рисков для нового проекта

№ 10.5.1.3 — Технический компромисс (Trade-off)

- **Что нужно знать:** как принимать решения, как аргументировать
- **Вопросы на собеседовании:** "Как вы принимаете архитектурные решения?"
- **Практика:** Задokumentировать trade-offs для архитектурного решения

№ 10.5.1.4 — RFC (Request for Comments)

- **Что нужно знать:** процесс согласования архитектурных решений
- **Вопросы на собеседовании:** "Что такое RFC?"
- **Практика:** Написать RFC на внедрение нового микросервиса

№ 10.5.1.5 — Стратегическое планирование

- **Что нужно знать:** roadmap на полугодие/год
- **Вопросы на собеседовании:** "Как вы планируете развитие системы?"
- **Практика:** Составить стратегический план развития системы на год

№ 10.5.1.6 — Технический долг

- **Что нужно знать:** как управлять, как доносить до продакта
- **Вопросы на собеседовании:** "Как убедить продакта выделить время на рефакторинг?"
- **Практика:** Подготовить презентацию о техническом долге для продакта

10.5.2. Бизнес-ориентированность (Staff Level)

№ 10.5.2.1 — Business Value

- **Что нужно знать:** как технические решения влияют на бизнес
- **Вопросы на собеседовании:** "Как ваше решение влияет на бизнес?"
- **Практика:** Сформулировать бизнес-ценность своего проекта

№ 10.5.2.2 — Коммуникация с бизнесом

- **Что нужно знать:** перевод бизнес-требований в технические задачи
- **Вопросы на собеседовании:** "Как перевести требования продукта в задачи?"
- **Практика:** Взять требование продукта и разложить на технические задачи

№ 10.5.2.3 — Cost Optimization

- **Что нужно знать:** оптимизация затрат на инфраструктуру
- **Вопросы на собеседовании:** "Как уменьшить затраты на инфраструктуру?"
- **Практика:** Проанализировать текущие затраты и предложить способы оптимизации

№ 10.5.2.4 — Time-to-Market

- **Что нужно знать:** как ускорить вывод продукта на рынок
- **Вопросы на собеседовании:** "Как ускорить разработку?"
- **Практика:** Предложить способы ускорить релиз новой функции

10.5.3. Практические задания (Staff Level)

№ 10.5.3.1

- **Задание:** Написать SLO для своего сервиса
- **Что проверяет:** Понимание NFR

№ 10.5.3.2

- **Задание:** Написать RFC на внедрение нового архитектурного решения
- **Что проверяет:** Знание процессов

№ 10.5.3.3

- **Задание:** Подготовить презентацию о техническом долге для продакта
 - **Что проверяет:** Коммуникация с бизнесом
-

РАЗДЕЛ 10.6: ОРГАНИЗАЦИОННОЕ ЛИДЕРСТВО (STAFF LEVEL)

10.6.1. Управление людьми (Staff Level)

№ 10.6.1.1 — Мотивация

- **Что нужно знать:** что мотивирует разработчиков (автономия, мастерство, цель)
- **Вопросы на собеседовании:** "Как мотивировать разработчиков?"
- **Практика:** Провести 1-on-1 и понять, что мотивирует каждого члена команды

№ 10.6.1.2 — Выгорание

- **Что нужно знать:** признаки, как предотвратить, как помочь
- **Вопросы на собеседовании:** "Как заметить и предотвратить выгорание?"
- **Практика:** Провести чек-ин на выгорание с каждым членом команды

№ 10.6.1.3 — 1-on-1

- **Что нужно знать:** как проводить эффективные встречи с подчиненными
- **Вопросы на собеседовании:** "Как проводить 1-on-1?"
- **Практика:** Провести 1-on-1 с членом команды и обсудить его состояние и планы

№ 10.6.1.4 — Онбординг

- **Что нужно знать:** план на первую неделю/месяц для новичка
- **Вопросы на собеседовании:** "Как организовать онбординг?"
- **Практика:** Составить онбординг-план для нового разработчика

№ 10.6.1.5 — Конфликты

- **Что нужно знать:** как разрешать в команде и между командами
- **Вопросы на собеседовании:** "Как разрешить конфликт в команде?"

- **Практика:** Разрешить конфликт между двумя разработчиками (реальный или вымышленный)

№ 10.6.1.6 — Инженерная культура

- **Что нужно знать:** как строить, как поддерживать
 - **Вопросы на собеседовании:** "Как строить инженерную культуру?"
 - **Практика:** Внедрить практику "no-blame" постмортемов
-

10.6.2. Инженерная культура (Staff Level)

№ 10.6.2.1 — Blame-less Post Mortems

- **Что нужно знать:** анализ инцидентов без поиска виноватых
- **Вопросы на собеседовании:** "Как писать постмортемы?"
- **Практика:** Написать blameless post-mortem на реальный или вымышленный инцидент

№ 10.6.2.2 — Continuous Improvement

- **Что нужно знать:** непрерывное улучшение процессов
- **Вопросы на собеседовании:** "Как вы улучшаете процессы в команде?"
- **Практика:** Предложить улучшение процесса после ретроспективы

№ 10.6.2.3 — Diversity & Inclusion

- **Что нужно знать:** как создавать инклюзивную среду
- **Вопросы на собеседовании:** "Как сделать команду более инклюзивной?"
- **Практика:** Провести анализ гендерного баланса в команде

№ 10.6.2.4 — On-call

- **Что нужно знать:** организация дежурств без выгорания
- **Вопросы на собеседовании:** "Как организовать дежурства?"
- **Практика:** Создать расписание дежурств с ротацией и компенсациями

№ 10.6.2.5 — Best Practices

- **Что нужно знать:** как внедрять и поддерживать лучшие практики
- **Вопросы на собеседовании:** "Как внедрить новый стандарт в команде?"

- **Практика:** Внедрить практику написания тестов для всех новых фич
-

10.6.3. Практические задания (Staff Level)

№ 10.6.3.1

- **Задание:** Написать blameless post-mortem на инцидент
- **Что проверяет:** Знание постмортемов

№ 10.6.3.2

- **Задание:** Провести 1-on-1 с членом команды и записать выводы
- **Что проверяет:** Управление людьми

№ 10.6.3.3

- **Задание:** Составить онбординг-план для нового разработчика
 - **Что проверяет:** Передача знаний
-

🔗 СВОДНЫЙ ЧЕК-ЛИСТ ДЛЯ ПОДГОТОВКИ К СОБЕСЕДОВАНИЮ

Junior Level (должен знать 100%)

Как написать README

Как проводить code review (базовые принципы)

Как документировать API

Как писать комментарии в коде

Middle Level (должен знать 100%)

ADR (Architecture Decision Record)

Модель SBI для фидбека

Технический долг (управление)

Code review: что проверять, как давать фидбек

Scrum/Kanban (базовые принципы)

Оценка задач (Story Points)

Senior Level (должен знать 100%)

Проведение собеседований

Составление тестовых заданий

Составление IDP для подчиненных

Менторство

Декомпозиция, roadmap

Управление рисками

Коммуникация с продактом

Staff Level (должен знать 100%)

NFR: SLA, SLO, SLI

RFC (Request for Comments)

Стратегическое планирование

Бизнес-ориентированность (business value, cost optimization)

Blameless post-mortems

1-on-1, мотивация, выгорание

Инженерная культура, diversity & inclusion

On-call организация

🛠 ПРАКТИЧЕСКИЙ ПРОЕКТ ДЛЯ ЗАКРЕПЛЕНИЯ ДОМЕНА 10

Задание: Стать техническим лидером для вымышленного проекта и пройти все этапы управления.

Шаги:

1. **Стратегия:** Составить roadmap на 6 месяцев для проекта
2. **Процессы:** Настроить Scrum/Kanban для команды
3. **Документация:** Написать ADR для архитектурного решения
4. **Code Review:** Провести code review и дать фидбек
5. **Найм:** Составить план собеседования и тестовое задание
6. **Развитие:** Составить IDP для каждого члена команды

7. **Коммуникация:** Провести 1-оп-1 с каждым членом команды
8. **Риски:** Провести анализ рисков и предложить mitigation
9. **Постмортем:** Написать blameless post-mortem на инцидент
10. **Бизнес:** Подготовить презентацию о бизнес-ценности проекта

Проверка:

- Все документы должны быть согласованы с "руководством" (самим собой)
- План должен быть реалистичным и выполнимым
- Команда должна быть мотивирована и понимать цели